

gPAC: Low-Overhead Control-Pointer Integrity for CUDA Device Code

Chaochao Zhang^{1,2*} and Xingbin Wang³

¹ School of Cyber Science and Engineering, Ningbo University of Technology, Ningbo, China

² State Key Laboratory of Information Security, Institute of Information Engineering, CAS, Beijing, China
`zhangchaochao@nbut.edu.cn`

³ School of Cybersecurity, Tianjin University, Tianjin, China
`wangxingbin@tju.edu.cn`

Abstract. Modern CUDA applications increasingly use indirect device control transfers, such as function-pointer dispatch tables and callback-like control pointers, to support runtime specialization, reusable device code, and performance-tuned execution. These control pointers become security-critical in the presence of GPU memory corruption: a corrupted dispatch entry or object virtual-table pointer can redirect device-side execution to unintended code. However, existing GPU memory-safety defenses either target broad classes of memory errors with substantial runtime cost or rely on hardware and toolchain support that is unavailable on commodity NVIDIA GPUs, leaving low-overhead control-pointer integrity for CUDA device code insufficiently addressed.

We present gPAC, a compiler-assisted system that provides low-overhead, software-enforced probabilistic integrity for selected CUDA device control pointers on commodity NVIDIA GPUs. gPAC protects memory-resident function pointers and object virtual-table pointers using compact authentication tags embedded in unused pointer bits, per-kernel-instance keys derived from hidden launch seeds, and authentication immediately before indirect control transfer. Under bounded online guessing and without reuse of a valid same-domain signed pointer, gPAC detects forged, corrupted, and cross-domain control pointers before control transfer, while explicitly not providing complete memory safety or protection against same-type pointer replay.

We implement gPAC in an LLVM 18-based CUDA toolchain. Across 18 CUDA workloads and 20 kernel-only runs per workload, gPAC incurs a median overhead of 3.59%, a mean overhead of 4.34%, and a geometric-mean slowdown of 1.042×

Keywords: CUDA · GPU security · pointer authentication · control-flow integrity · HPC

* Corresponding author.

1 Introduction

Graphics processing units (GPUs) are now a primary execution substrate for high-performance computing (HPC), scientific simulation, graph analytics, and machine-learning workloads. CUDA kernels are no longer simple straight-line numeric loops; many applications include device-side control logic, operation selection, callback-style computation, and hand-written dispatch structures. A common implementation pattern is a function-pointer dispatch table: a kernel chooses a device function according to runtime data and invokes it through an indirect call.

This flexibility creates a control-data attack surface. CUDA is extension of the C/C++ programming languages and therefore inherits their memory-safety vulnerabilities, including out-of-bounds writes, stale-pointer accesses, and buffer overflows. Prior studies have demonstrated that CUDA memory corruptions can overwrite function pointers or C++ virtual dispatch metadata and redirect GPU execution [1, 2]. More recent work has shown that modern GPUs support practical exploitation techniques including code injection and code-reuse attacks [3]. These results suggest that protecting device-side control pointers is important for GPU-accelerated HPC applications.

Unfortunately, existing protection mechanisms leave a gap. Full memory-safety or bounds-checking systems can detect many root-cause errors but may be too broad or expensive for production HPC kernels. Binary-level control-flow integrity (CFI) can constrain indirect branches but may lose source-level type information and does not directly protect the memory objects that store code pointers. CPU-side pointer authentication, such as ARM PAC, is not available as a hardware feature for NVIDIA CUDA device code. This motivates a software mechanism that is targeted, compiler-assisted, and optimized for the GPU execution model.

We propose gPAC, a low-overhead software pointer-authentication system for selected CUDA device control pointers. gPAC protects the pointer object itself: before a function pointer or virtual-table pointer is stored in memory, gPAC signs it by embedding a truncated tag into unused high bits of the pointer; before an indirect device call or virtual dispatch consumes the pointer, gPAC authenticates the tag and traps on mismatch. Authentication keys are derived from hidden per-kernel-instance launch seeds and static pointer categories or function-pointer type identifiers. Under the stated threat model, this design causes raw, forged, or cross-domain control-pointer overwrites to fail authentication before control transfer, while deliberately preserving ordinary pointer-copying semantics in the default compatibility mode; as a result, same-type signed-pointer replay is an explicit boundary rather than a claimed defense.

We implement gPAC in an LLVM 18-based CUDA toolchain and evaluate it on 18 CUDA workloads. Across 20 kernel-only runs per workload, gPAC incurs a median overhead of 3.59%, a mean overhead of 4.34%, and a geometric-mean slowdown of $1.042\times$. Security experiments further demonstrate that corrupted function pointers and virtual-table pointers are detected before the corresponding indirect control transfer.

Contributions. This paper makes three contributions.

1. We formulate and design a software-enforced integrity mechanism for selected CUDA device control pointers. gPAC protects device function pointers and object-resident vptrs at the point where memory corruption would otherwise become an indirect device control transfer. The security goal is explicitly bounded: gPAC provides probabilistic fail-fast detection of raw, forged, and cross-domain pointer overwrites under hidden per-launch keys, while excluding complete memory safety, hardware-equivalent PAC guarantees, and same-type replay prevention in the default compatibility mode.
2. We show how to realize this protection on commodity NVIDIA GPUs without changing CUDA pointer layouts. The LLVM 18/NVVM implementation embeds compact tags in unused high pointer bits, derives per-kernel and per-type authentication state from hidden launch seeds, instruments function-pointer and virtual-dispatch flows, and exposes configurable handling for unsupported or unknown pointer flows.
3. We evaluate the security boundary, compatibility, and cost of the design. Security harnesses show fail-fast behavior for forged function-pointer and vptr overwrites and expose same-type replay as an intentional limitation. Across 18 CUDA workloads and 20 repeated kernel-only runs per workload and configuration, gPAC compiles and runs all workloads and incurs 3.59% median overhead with a $1.042\times$ geometric-mean slowdown.

2 Background and Motivation

2.1 CUDA Device Control Pointers: Dispatch Tables and Virtual Tables

CUDA supports device functions and, in restricted forms, device-side function pointers and indirect calls. A dispatch table is an array of function pointers used to select an implementation at runtime. Such tables are a natural implementation technique for runtime operation selection, physics-model selection, graph-processing operators, and small device-side interpreters. They let a kernel choose among device implementations without returning to the host or launching a separate kernel for each case.

CUDA C++ also supports restricted forms of device-side virtual dispatch. For classes with virtual device methods, the compiler emits device-side virtual tables and stores a virtual-table pointer in each polymorphic object. A virtual call on the device is lowered to a sequence that loads the object’s virtual-table pointer, loads the selected table entry, and performs an indirect call.

Virtual dispatch is vulnerable to a different but closely related corruption path. The virtual tables themselves are normally emitted as compiler-managed program objects and are harder to modify directly. However, the objects that carry virtual-table pointers are ordinary device objects and may reside in writable memory such as global memory, heap allocations, or stack-like per-thread storage. Existing memory-safety errors, including use-after-free bugs and spatial

overflows, can therefore overwrite an object’s virtual-table pointer and make it point to an unintended or attacker-chosen table address within the reachable corruption model. The next virtual call through that object then loads a table entry from the corrupted vptr target and may transfer control to an unintended device target.

From a security perspective, hand-written dispatch tables and compiler-generated virtual tables expose the same kind of control data: memory-resident function pointers that determine the target of a later device-side indirect call. They differ in how they are produced and reached. A dispatch table is explicitly allocated and indexed by the programmer, whereas a virtual table is created by the C++ compilation model and reached through an object virtual-table pointer. In both cases, corrupting the table entry or the pointer used to reach the table can redirect device control flow to an unintended device function or gadget sequence. gPAC focuses on this class of CUDA device control pointers.

2.2 Device-Side Memory Corruption as a Control-Flow Threat

Prior work demonstrates that GPU memory bugs can be exploitable. Miele showed that CUDA buffer overflows can corrupt data and control objects, including function pointers and C++ vtables [1]. Di et al. studied overflow vulnerabilities on GPUs and discussed function-pointer overwrites [2]. Guo et al. later demonstrated modern GPU memory exploitation techniques, including code-injection and code-reuse attacks on NVIDIA GPUs [3]. These works motivate treating CUDA device memory corruption as a security problem rather than only a correctness problem.

Figure 1 shows a simplified kernel-lifetime dispatch table. Each block uses a table in global memory. A bounded out-of-bounds write corrupts a nearby table entry. In an unprotected baseline, the corrupted pointer is consumed by an indirect call.

gPAC changes the table entries from raw function pointers to authenticated protected pointers. If the attacker writes an arbitrary target address without a valid tag, authentication fails and the kernel traps before the indirect call.

2.3 Design Goals

gPAC is guided by five goals.

1. **Low overhead.** Authentication must be cheap enough for performance-sensitive GPU kernels.
2. **Pointer-layout compatibility.** Protected pointers should remain 64-bit values so arrays, tables, and structs do not become fat-pointer objects.
3. **Compiler automation.** Programmers should not manually wrap every device function-pointer assignment and indirect call.
4. **Type-sensitive protection.** Function pointers of different static types should not be interchangeable.
5. **Fail-fast semantics.** A protected control pointer with an invalid tag should terminate the kernel before control transfer.

<pre> 1 class Dummy(){ 2 __device__ virtual void vFUNC(char* input, int length) 3 }; 4 5 __global__ void virtual_dispatch_kernel(char* input, int 6 int idx = blockDim.x * blockDim.x + threadIdx.x; 7 char* buf = (char*)cummalloc(sizeof(char)*BUF_LEN); 8 Base*obj = new Base(); 9 /*Attacker overwrites the class V-table, 10 which is stored in first 64-bit of obj*/ 11 memcpy(buf, inout, length); 12 //control flow hijacked 13 obj->vFUNC(input+(length*idx), length); 14 } </pre>	<pre> 1 using Fn = void (*)(char*, int); 2 __device__ void dFunc1(char* input, int length) { ...}; 3 __device__ void dFunc2(char* input, int length) { ...}; 4 5 __global__ void function_dispatch_kernel(char* input, int 6 int idx = blockDim.x * blockDim.x + threadIdx.x; 7 Fn dispatch_table[] = {dFunc1, dFunc2}; 8 char buf[BUF_LEN] 9 /*Attacker overwrites the dispatch function table 10 Via controlled input and length*/ 11 memcpy(buf, inout, length); 12 //control flow hijacked 13 dispatch_table[idx](input+(length*idx), length); 14 } </pre>
---	---

Fig. 1. Motivating dispatch-table attack and protection point. A memory-corruption bug can overwrite a device function pointer stored in a dispatch table; gPAC signs protected table entries before storage and authenticates the loaded pointer before the indirect device call, causing corrupted values without a valid tag to fail before control transfer.

3 Threat Model and Security Goals

Trusted components. We trust the host application binary, the gPAC compiler pass, the gPAC runtime and launch wrapper, the CUDA driver/runtime, and the integrity of the compiled kernel code. Protecting kernel binaries from tampering is orthogonal to gPAC.

Attacker model. The attacker controls host-provided inputs and can trigger CUDA device memory-safety bugs that yield bounded or layout-constrained writes, such as out-of-bounds or stale writes, to application data, dispatch tables, or vptr-bearing objects. The attacker may read limited nearby or reachable device memory and knows gPAC’s algorithms, pointer layout, tag width, and instrumentation sites. The attacker cannot modify host code, runtime wrappers, PTX/SASS, compiler-inserted checks, the CUDA driver, or GPU hardware. The attacker also cannot arbitrarily inspect device memory, kernel-parameter storage, registers, or spilled local state in a way that reveals hidden launch seeds or derived per-type keys. This seed/key secrecy assumption is a deployment boundary of software pointer authentication on commodity GPUs and is weaker than hardware PAC key isolation.

Security goal and scope. gPAC aims to stop memory-corruption-induced hijacking of protected CUDA device function pointers and object-resident vptrs when the attacker writes a raw, forged, or cross-domain control-pointer value. It authenticates protected function pointers immediately before indirect device calls and protected vptrs before virtual-dispatch slot loads, trapping on mismatch. Tag guessing is treated as a bounded online attack because failed authentication terminates the kernel instance; relaunch-based retry resistance depends on fresh

launch seeds or host-side rate limiting. gPAC should therefore be read as a low-overhead probabilistic mitigation for selected control-pointer corruptions, not as a proof of cryptographic unforgeability or as complete GPU memory safety. gPAC does not stop data-only attacks, protect unrelated control-flow mechanisms, defend after key disclosure, prevent same-type signed-pointer replay in the default compatibility mode, or enforce exact C++ class-hierarchy CFI for vptrs.

Short-tag interpretation. The in-pointer tag is deliberately compact so that protected values remain ordinary 64-bit CUDA pointers. This design point is similar to Arm MTE and HWASan, which use short pointer tags to turn many memory-safety violations into probabilistic fail-stop events, and to Arm PAC, which embeds a truncated authentication code in otherwise unused pointer bits. The security interpretation is correspondingly bounded: an N -bit tag gives a per-attempt random-forgery probability of approximately 2^{-N} when the attacker lacks the relevant key and cannot reuse an already valid same-domain signed pointer. After q independent online attempts, the success probability is

$$P_{forge}(N, q) = 1 - (1 - 2^{-N})^q.$$

For example, 12-bit tags give success probabilities of about 2.4% after 100 independent attempts and 21.7% after 1000 attempts, while 16-bit tags reduce these to about 0.15% and 1.5%, respectively. The 8-bit mode is therefore a degraded compatibility fallback rather than a recommended setting for adversarial deployments with repeated attempts. In the intended deployment, failed authentication terminates the current kernel, fresh launch seeds make guesses from one kernel instance non-transferable to the next, and services that expose repeated attacker-controlled launches should combine gPAC with ordinary crash handling or rate limiting.

4 gPAC Design

4.1 Design Overview

For protected control-pointer classes within the instrumented subset, gPAC enforces an authenticated-control-pointer invariant: a selected CUDA device function pointer or object-resident vptr may be stored in memory only in protected form, and it must be authenticated immediately before it is interpreted as a control-flow target or as the base of a virtual-call table lookup. For ordinary device function pointers, the compiler signs the target before storing it into a table, struct field, or other protected location, and authenticates the loaded value before an indirect device call. For C++ virtual dispatch, the protected value is the object-resident virtual-table pointer (vptr); gPAC authenticates the vptr before using it to load the virtual-call target from the compiler-emitted vtable. A small runtime supplies hidden per-kernel-instance launch seeds and chooses the in-place tag width. Figure 2 summarizes the compile-time and runtime protection flow.

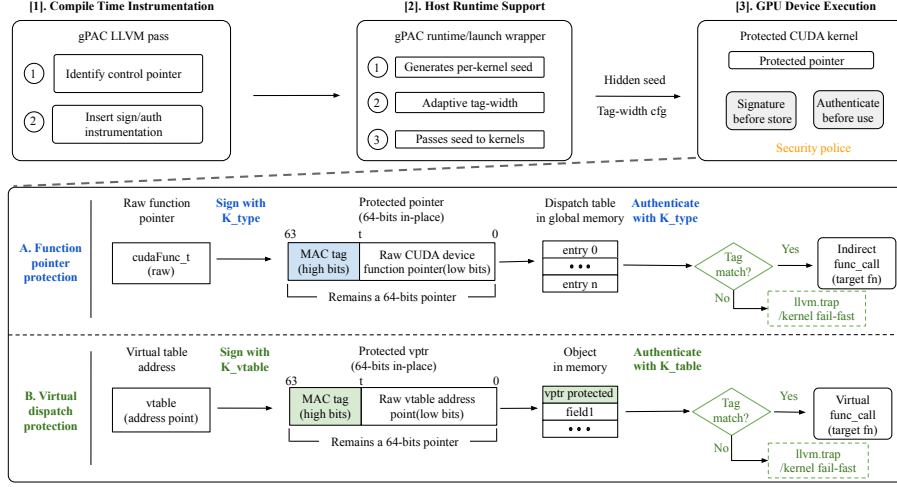


Fig. 2. Overview of the gPAC protection workflow. The compiler inserts signing before protected control pointers are stored and authentication before indirect calls or virtual dispatch. The runtime supplies hidden per-kernel-instance launch seeds and tag-width configuration, and authentication failures trap before control transfer.

4.2 Pointer Encoding and Authentication

gPAC represents each protected control pointer as an in-place 64-bit value:

$$protected_fp = raw_fp \mid (tag \ll raw_pointer_bits).$$

The low bits hold the raw CUDA device function pointer or vtable address point, and the high bits hold a truncated MAC tag. This representation preserves the size of pointer-bearing tables, objects, and structs, which is important for CUDA code that copies or indexes those objects directly. Authentication masks out the high bits to recover the raw pointer, recomputes the expected tag under the appropriate key, compares tags, and traps on mismatch.

CUDA virtual-address layouts vary across GPU generations, driver versions, and pointer types. gPAC therefore probes pointer layouts at runtime. The probe samples `cudaMalloc` device pointers and device function pointers, computes the common unused high-bit range, and selects a conservative process-wide tag width. Preferred configurations target 12–16 bits; an 8-bit mode is a degraded-security fallback. If fewer than 8 high bits are available, in-place tags are unsupported on that platform. The resulting 2^{-N} forging bound should be read as a truncated-tag online guessing bound under hidden keys and the selected MAC assumption, not as an unconditional cryptographic proof.

gPAC uses *gPAC-fast* as its default authentication primitive: a lightweight multiply-xor-rotate keyed tag-generation function designed to make protected control-pointer corruption fail fast at low GPU cost. Given a raw control pointer

P , the derived key K_T , and the corresponding domain identifier D_T , gPAC-fast mixes these values using integer multiplication, XOR, and fixed-width rotation operations and then truncates the result to the configured tag width N :

$$tag = Trunc_N((P, K_T, D_T)).$$

All operations are implemented using integer instructions supported by CUDA device code and require no additional memory accesses for tag generation.

For the security analysis, we model the keyed output of gPAC-fast as a pseudorandom N -bit value for attacker-chosen inputs when the corresponding key remains secret. Under this assumption, a random forgery succeeds with probability approximately 2^{-N} per independent online attempt. gPAC-fast is not intended to provide cryptographic unforgeability; rather, it provides a performance-oriented probabilistic integrity mechanism for protected control pointers. The implementation also includes experimental stronger backends, including SipHash-like, polynomial, QARMA-like, GMAC-like, and AES-like variants, to expose the cost of replacing the default performance-oriented primitive with more instruction-heavy authentication logic. Unless otherwise stated, the main performance claims correspond to gPAC-fast; deployments that require a more conservative authentication assumption can choose a stronger backend at the overheads reported in Section 6.

4.3 Key Derivation and Type Separation

Each protected kernel instance receives a hidden 128-bit launch seed:

$$seed_{kernel} = (seed_{lo}, seed_{hi}).$$

At kernel entry, gPAC derives one key per static function-pointer type and one key domain for protected vtable pointers:

$$K_T = KDF(seed_{kernel}, type_id_T).$$

For ordinary function pointers, this provides type-sensitive authentication: a signed pointer for type T_1 should not authenticate at a call site expecting type T_2 except with the truncated-tag guessing probability for the selected tag width and MAC mode. The current vtable-pointer domain distinguishes vptrs from ordinary function pointers, but it does not distinguish individual C++ classes or class hierarchies. Consequently, a valid signed vptr from another class in the same protected vptr domain is a replay/substitution boundary of the current design rather than a prevented attack. More precise class-hierarchy type identifiers require frontend metadata and are discussed in Section 8. The KDF reuses lightweight integer mixing and runs at kernel entry, not on every indirect call.

4.4 Instrumentation of Dispatch Tables and Virtual Calls

Given the encoding and keys above, the compiler enforces two rules: sign when a protected control pointer is produced for storage, and authenticate at the last

point before the protected value drives control flow. This rule is applied to both explicit dispatch tables and C++ virtual dispatch, but the protected memory object differs in the two cases.

For mutable device function-pointer tables, the current gPAC implementation targets kernel-lifetime dispatch tables. A protected dispatch table is initialized inside a kernel or a device function reached by that kernel, used during the same kernel execution, and logically invalid after the kernel terminates. We do not target mutable long-lived global device function-pointer tables in the current implementation. For evaluation, we focus on per-block dispatch tables in global memory. A designated block thread initializes and signs entries, then synchronizes the block before use:

C++ virtual calls use a different memory object from hand-written dispatch tables. The call target normally resides in a compiler-emitted vtable, while each object stores a `vptr` that points to the appropriate vtable address point. gPAC protects this object-resident `vptr`. When the compiler sees a store of a vtable address point into an object, it signs the vtable address point under the vtable-pointer domain before the store. Before a virtual call performs the vtable slot load, gPAC authenticates the object `vptr` and uses the recovered raw vtable address point for the slot access.

This design protects against memory corruptions that overwrite an object’s `vptr` with an arbitrary or cross-domain pointer. It intentionally does not sign each vtable entry: CUDA C++ vtables are compiler-generated global objects, and the current design treats them as immutable code metadata. If an attacker can rewrite vtable memory itself, that capability is outside the current threat model.

4.5 Compatibility, Replay, and Failure Semantics

The default MAC input is:

$$tag = Trunc_N(MAC_{K_T}(raw_fp)).$$

This compatibility mode does not bind tags to storage addresses. It preserves ordinary pointer-copying, struct-copying, and table-copying semantics, which is important for CUDA programs that move tables and objects through existing data structures. The trade-off is that same-kernel, same-type signed-pointer replay is not prevented. A strict mode could include the storage address in the MAC input, but this can break common copying patterns; we leave it as future work.

The compiler can precisely classify many IR values as raw, signed, or authenticated, but low-level CUDA code can also move pointers through opaque casts, integer conversions, or uninstrumented memory. gPAC therefore exposes an unknown-flow policy. In *allow* mode, unknown flows are recorded as metadata and compilation proceeds. In *warn* mode, the compiler emits diagnostics for unknown function-pointer flows. In *error* mode, the compiler rejects such code. This makes the system usable both as a compatibility-oriented compiler mode and as a stricter security-auditing tool.

Authentication failure lowers to `llvm.trap` and terminates the offending kernel. The host observes the failure through standard CUDA error reporting, such as `cudaDeviceSynchronize` or `cudaGetLastError`. An optional debug mode records compact metadata such as callsite ID, type ID, expected tag, actual tag, and thread/block coordinates before trapping.

5 Implementation

5.1 Compiler Instrumentation

We implement gPAC as compiler instrumentation over CUDA device IR in an LLVM 18-based toolchain. The implementation has two responsibilities: it must place signing and authentication at the control-pointer production/use boundary described in Section 4, and it must avoid silently treating unsupported low-level pointer flows as protected. The instrumentation is enabled through the `-fpacgpu` compilation option, allowing the same benchmark sources to be built as either baseline or gPAC binaries.

For device function pointers, the pass signs raw control pointers when they become persistent program values, including function-address creation, dispatch-table initialization, memory stores, and function-pointer returns. It authenticates only at the final use site before an indirect device call, keeping overhead proportional to protected control transfers rather than ordinary pointer loads.

For C++ virtual dispatch, the pass recognizes vtable address points in device IR and instruments stores of those address points into object `vptr` fields. At a virtual-call site, it authenticates the loaded `vptr` before the vtable slot load and uses the recovered raw `vptr` for the slot access. This matches the design in Section 4: gPAC protects the mutable `vptr` stored in the object, not the immutable compiler-emitted vtable entries.

To avoid overclaiming coverage, the pass tracks function-pointer SSA values as *Raw*, *Signed*, or *Unknown*. Unknown values arise from opaque casts, `ptrtoint/inttoptr` propagation, incompatible function-pointer casts, or uninstrumented producers. Depending on policy, the compiler records these flows, warns about them, or rejects the compilation unit. NVPTX inline assembly is explicitly excluded from indirect-call authentication because LLVM represents it as a call-like target even though it is not a CUDA function-pointer transfer.

5.2 Runtime and Launch Support

The runtime completes the compiler transformation by supplying the values that are unavailable in device IR alone. It generates per-kernel-instance launch seeds, passes them through hidden kernel parameters, runs the pointer-layout probe, selects the process-wide tag width, and provides optional debug metadata buffers. The host application and launch wrapper are trusted in the threat model.

The implementation closes the launch-seed loop by rewriting CUDA host launch paths to use gPAC launch-wrapper logic. The wrapper supplies the hidden

seed parameters expected by instrumented kernels while preserving the source-level benchmark interface. This lets the same CUDA source code run in either baseline or gPAC mode, with the protection boundary determined by the compiler option and launch wrapper rather than by manual application changes.

Seed and key exposure. The launch seed is not stored in protected pointer objects; it is delivered through compiler-added kernel parameters and mixed into per-type keys at kernel entry. As stated in the threat model, gPAC assumes that the attacker cannot arbitrarily inspect parameter storage, registers, or spilled local state that contains these seeds or derived keys. This is a software deployment assumption rather than hardware PAC-style key isolation. A disclosure that reveals the seed, a derived key, or enough signed pointer/tag pairs to attack the selected non-cryptographic backend invalidates the forgery bound. Debug metadata that records expected and actual tags is therefore intended for testing and is outside the production threat model.

5.3 Implementation Scope and Validation

The implementation includes the compiler pass, Clang driver integration, run-time launch-wrapper support, seed propagation, unknown-flow policies, vptr support, and benchmark scripts for paired baseline and gPAC builds. For debugging and lit-level verification, the pass annotates transformed IR with audit metadata for signed stores, authenticated calls, unknown flows, launch-wrapper rewriting, vptr signing, vptr authentication, and recovered raw callees. These metadata nodes are not part of the security mechanism; they make the compiler transformation auditable.

The lit suite tests function-pointer signing and authentication, metadata behavior, unknown-flow policy behavior, native launch-wrapper rewriting, seed propagation, vptr authentication, and inline-assembly exclusion. The gPAC lit tests pass, and all 18 selected CUDA workloads compile and run in both baseline and gPAC configurations.

6 Evaluation

The evaluation follows the two promises made by the design. First, gPAC should stop corrupted device function pointers and object vptrs before they reach indirect calls or virtual dispatch, while preserving the replay limitation stated in the threat model. Second, by keeping protected pointers in place and authenticating only near control transfer, gPAC should add modest kernel-time overhead on CUDA workloads. We also check the deployment assumptions behind the implementation: whether the benchmark set runs in both build modes, what low-level binary effects explain the overhead, how alternative MAC backends affect overhead, and how many high pointer bits are available on the evaluation platform.

6.1 Experimental Setup

The evaluation uses 18 CUDA workloads and two build configurations on one multi-GPU server. Ten workloads come from graph, simulation, and stencil-style benchmark directories, including BFS, connected-components, PageRank, generation, Game-of-Life, stencil, and traffic kernels. Eight additional workloads stress virtual dispatch or function-pointer dispatch, including Dragon-style AMR, BFS, join, and SSSP kernels, plus graph relaxation, vector-add, pathfinder, and stencil kernels. The platform is an Ubuntu 24.04.2 server with eight NVIDIA GeForce RTX 4090 D GPUs, NVIDIA driver 550.144.03, CUDA 12.9 build tools, and an LLVM/Clang 18.1.8 CUDA toolchain targeting `sm_89`.

The main comparison builds each workload twice from the same source: a baseline using the gPAC-capable LLVM toolchain without `-fpacgpu`, and gPAC-fast using `-fpacgpu` with the lightweight multiply-xor-rotate MAC. Both configurations use `-O3`, disable LTO/offload LTO, and disable whole-program virtual-function elimination options that would remove the dispatch behavior under study. We report source-level CUDA event time around the main algorithm kernels only; initialization, object construction, I/O, copyback, and checksum kernels are excluded. The formal run uses 20 repetitions per workload and configuration, with each baseline/gPAC pair placed on the same GPU worker.

6.2 Security Evaluation

gPAC targets control-pointer integrity rather than complete memory safety. We therefore test whether corrupted control-pointer values are rejected where CUDA would otherwise consume them as indirect-call or virtual-dispatch operands. Each CUDA harness initializes a valid control-flow object, overwrites the stored function pointer or `vptr` bits, and consumes the corrupted object in a one-thread kernel. Table 1 reports the observed CUDA status and sentinel return values for baseline and gPAC builds.

The results match the intended boundary. Baseline CUDA reaches unsafe targets after dispatch-table, scalar function-pointer, cross-type, and `vptr` overwrites, whereas gPAC traps before those corrupted target values are returned. Same-type replay remains executable because the default mode does not bind tags to storage addresses; a signed cross-class `vptr` replay would exercise the same boundary because the current design uses one `vptr` domain rather than class-specific `vptr` keys. These harnesses are not intended to demonstrate new end-to-end GPU exploits. Instead, they isolate the security property that gPAC claims: corrupted protected control-pointer values are rejected at the use-before-control-transfer point under the stated assumptions.

6.3 Performance Evaluation

To evaluate whether targeted control-pointer integrity is practical for CUDA kernels, we measure the cost of protected control-pointer production and use rather than full memory-access checking. This measurement corresponds to the

Table 1. Security harness results. “Unsafe” means that the corrupted target executed and returned its sentinel value; “Fail fast” means that the kernel raised a CUDA failure before a corrupted target value was returned. Outcomes describe the observed behavior under the threat model and harness conditions.

Attack harness	Baseline result	gPAC result	Outcome
Dispatch-table entry over-write	Unsafe target executed (901)	exe- Fail fast	Rejected before transfer
Scalar function-pointer overwrite	Unsafe target executed (902)	exe- Fail fast	Rejected before transfer
Cross-type substitution	Wrong-type target executed (903)	Fail fast	Rejected before transfer
vptr overwrite	Wrong virtual target executed (904)	tar- Fail fast	Rejected before transfer
Same-type replay	Replayed target executed (905)	Replayed target executed (905)	Boundary case
Invalid tag forging	Invalid program counter	Fail fast	Negative case

security design point of gPAC: protecting selected device control pointers at use-before-control-transfer points while preserving ordinary CUDA pointer layouts. Figure 3 reports per-workload overheads. Across all 18 workloads, the median overhead is 3.59%, the arithmetic mean overhead is 4.34%, and the geometric mean slowdown is $1.042\times$. The ten larger graph/simulation/stencil workloads have a 5.45% median overhead and $1.059\times$ geometric-mean slowdown; the eight virtual-dispatch/function-pointer workloads have a 1.88% median overhead and $1.022\times$ geometric-mean slowdown.

The results support the expected cost model: overhead is governed by the density of protected control-pointer operations relative to ordinary computation. Graph workloads exhibit the highest overheads: connected components, BFS, and PageRank variants range from 6.99% to 12.43%. Longer-running simulations such as Game-of-Life, stencil, and traffic remain lower, from 1.36% to 4.74%, because their main loops amortize authentication with more computation. The virtual-dispatch microbenchmarks mostly remain below 4%, supporting the choice to authenticate the object vptr once before the vtable slot load. The Dragon join outlier reports 14.72% overhead, but the absolute increase is only 0.003 ms; the two negative overheads also occur only in sub-millisecond kernels and fall within expected CUDA-event and code-generation noise.

Static binary metrics. Static cubin metrics collected from cuobjdump resource-usage and SASS dumps show the expected binary-level effects. gPAC increases SASS instruction records in every measured binary and usually adds 16–20 bytes of constant-memory state for injected authentication code and per-kernel key material. Register pressure is often unchanged for graph kernels but rises in some object-heavy workloads; for example, Game-of-Life increases from 34 to 50 maximum registers and the Dragon SSSP workload from 30 to 34. A separate

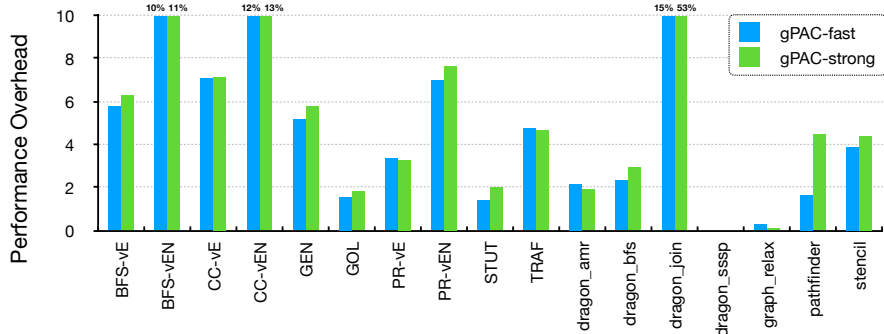


Fig. 3. Performance overhead per workload.

wall-clock timing pass confirmed that all 18 baseline and 18 gPAC whole-program executions completed, but those single-sample host times include initialization, I/O, and scheduling effects and are not used as the formal performance metric.

6.4 Compatibility and Sensitivity

The remaining experiments test implementation assumptions that affect deployability. They check whether the same benchmark sources run in both build modes, how alternative MAC backends affect overhead, and whether the in-place tag representation is feasible on the measured CUDA platform.

Benchmark compatibility. The implementation builds each benchmark twice from the same source tree: once with the gPAC-capable LLVM toolchain but without `-fpacgpu`, and once with `-fpacgpu` and the gPAC runtime linked in. Both configurations compile and run all 18 workloads in the current test environment. This compatibility result depends on two implementation details that are important for real CUDA code: `vptr` authentication for C++ virtual dispatch, and explicit exclusion of NVPTX inline assembly from indirect-call authentication.

MAC modes. The main performance results use gPAC-fast, so they should not be interpreted as the cost of heavier authentication primitives or as a claim that the default backend is cryptographic. To quantify this design choice, we ran a MAC-mode ablation on the same 18 workloads. Each mode was compiled from the same sources and measured with 10 CUDA-event repetitions per workload on GPU 0. All six backends compiled and ran all 18 workloads. Figure 4 reports overheads against the same baseline kernel times. We view these modes as a deployment spectrum: gPAC-fast is the performance-oriented compact-tag configuration, while the more instruction-heavy backends are appropriate when a deployment requires a stronger MAC assumption and can tolerate higher overhead.

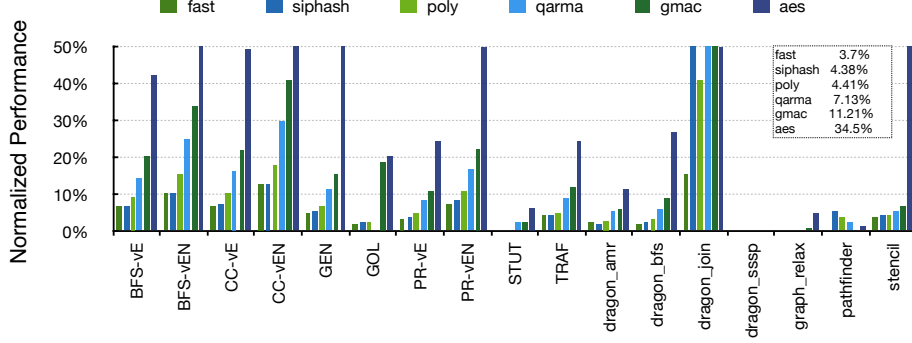


Fig. 4. Kernel-time overhead under different MAC backends. The ablation uses 18 workloads and 10 CUDA-event repetitions per workload on GPU 0.

The ablation confirms that gPAC’s overhead is sensitive to the authentication primitive. gPAC-fast has the lowest overall cost, with 3.70% median overhead and $1.041\times$ geometric-mean slowdown, which is close to the 20-run formal gPAC-fast result and supports its use as the default low-overhead configuration. SipHash-like and polynomial modes remain moderate, with 4.38% and 4.41% median overhead, respectively. QARMA-like and GMAC-like modes are higher at 7.13% and 11.21%, while the AES-like backend is substantially more expensive, reaching 34.59% median overhead and $1.424\times$ geometric-mean slowdown. These results expose a security/performance trade-off rather than changing the paper’s main performance claim: stronger or more instruction-heavy MACs may be useful for conservative deployments, but they are not free replacements for the default gPAC-fast mode. Conversely, the default gPAC-fast numbers should be interpreted as the cost of a practical probabilistic mitigation, not as the cost of cryptographic pointer authentication.

Adaptive tag width. The expected per-attempt random-forgery probabilities for 8-, 12-, and 16-bit tags are 2^{-8} , 2^{-12} , and 2^{-16} respectively. We measured high-bit availability on the evaluation node using a CUDA pointer-layout probe that samples CUDA Runtime API allocations, CUDA Driver API allocations, and device-side function-pointer representations. On the measured RTX 4090 D platform, the combined observed pointer representation used bits up to bit 46, leaving 17 high zero bits in the sampled values. A 12-bit tag placed in bits 63–52 is therefore compatible with the measured samples. This is a measured-platform result, not a cross-platform portability claim; other GPU generations, driver branches, MIG configurations, and future CUDA releases require separate validation.

7 Related Work

GPU exploitation and memory safety. Prior work shows that CUDA memory corruption can affect control data rather than only numerical results. Early studies demonstrated CUDA buffer overflows that corrupt function pointers or C++ virtual-table metadata [1, 2], while recent work shows practical GPU code-injection and code-reuse attacks on NVIDIA GPUs [3, 4]. GPU memory-safety tools and designs, including NVIDIA Compute Sanitizer, clARMOR, GPUShield, cuCatch, lock-and-key metadata, implicit memory tagging, and hardware/software co-designs, detect or prevent broader classes of invalid memory accesses [5, 6, 8–12]. gPAC is complementary: it does not provide complete spatial or temporal memory safety, but protects selected memory-resident control pointers at use-before-control-transfer points.

GPU CFI. GPU control-flow integrity can also be enforced at the binary-control-transfer layer. WarpGuard, the closest related system, operates on NVIDIA SASS binaries and validates protected control-flow sites after PTX lowering and SASS scheduling [13]. gPAC addresses a different layer: it signs function pointers and vptrs when they are created or stored and authenticates them before indirect device calls or virtual-dispatch slot loads. This retains source-level type information and launch-seed delivery, but does not cover arbitrary computed branches, return continuations, or inline assembly. The approaches are complementary: binary CFI constrains transfers at protected sites, while gPAC detects corruption while the control pointer is still a memory object.

CPU CFI, CPI, and pointer authentication. Classic CFI constrains indirect transfers to a statically derived control-flow graph [14]; compiler and binary systems refine this idea for forward edges and type-based policies [15–17]. CPI protects code pointers by isolating them in a safe region [18], and CCFI cryptographically protects control-flow-relevant values using CPU support such as AES-NI [20]. Arm Pointer Authentication and its compiler uses authenticate pointers with architectural PAC instructions [21–23], while HWASan, Arm MTE, and Intel LAM illustrate the broader use of high pointer bits for metadata [26–28]. gPAC borrows the principle that code-pointer objects deserve direct integrity checks, but implements software MACs in CUDA device code, derives keys from per-kernel launch seeds, and stores truncated tags in unused high bits of CUDA function pointers and vptrs. Like MTE and HWASan, gPAC trades tag width for layout compatibility and low overhead; unlike hardware PAC, it cannot rely on architecturally isolated keys or dedicated PAC instructions. Its guarantee is therefore best understood as software-enforced, bounded probabilistic control-pointer integrity for CUDA rather than as a hardware-equivalent PAC mechanism.

Virtual dispatch and GPU pointer representations. CPU C++ defenses such as SafeDispatch, LLVM CFI-vcall, VTrust, CFIXX, and VScape show that vptr corruption is a forward-edge control-flow threat and that class-hierarchy precision

matters [29, 16, 31–33]. gPAC does not claim full class-hierarchy CFI for CUDA C++; it protects the object-resident vptr before the vtable is dereferenced. CUDA’s support for device-side function pointers and virtual dispatch [34], and prior GPU virtual-function optimization work on object layout and pointer/type representations [35], show that pointer representation is a performance-critical GPU design space. gPAC uses that design space for targeted control-pointer integrity rather than dispatch optimization.

8 Limitations and Discussion

Replay, key disclosure, and online attempts. The default compatibility mode does not bind a tag to the storage address. An attacker who can read and copy a valid signed pointer of the same kernel and static type may replay it into another same-type slot. Address binding could prevent this, but may break pointer-copying, struct-copying, and table-copying semantics used by CUDA programs. gPAC also assumes hidden seeds and derived keys are not disclosed; once a key is leaked, software pointer authentication cannot prevent tag forgery. Short tags further mean that gPAC is not suitable as a stand-alone defense for deployments that give an attacker unlimited crash-observable retries. Fresh per-launch seeds make failed guesses non-transferable across independent kernel instances, but service-level retry resistance ultimately depends on ordinary crash handling, isolation, or rate limiting.

Tag and virtual-dispatch granularity. Short tags provide probabilistic protection only. The 8-bit fallback is a degraded-security configuration, and our tag-width result covers one RTX 4090 D platform rather than all GPU generations, MIG configurations, or future CUDA releases. For virtual dispatch, gPAC authenticates vptrs under a vtable-pointer domain and assumes compiler-emitted vtable globals are immutable. It prevents arbitrary raw vptr corruption under the threat model, but does not yet enforce class-specific or hierarchy-specific vptr integrity.

Coverage boundaries. gPAC protects selected function-pointer and vptr-mediated control flows, not complete memory safety. It does not prevent the underlying out-of-bounds write, use-after-free, data-only corruption, or attacks that patch kernel code or remove instrumentation. Long-lived global device function-pointer tables require module-stable keys or re-signing at kernel entry, and NVPTX inline assembly is intentionally excluded because treating inline PTX as a function-pointer target breaks legitimate CUDA library code.

Measurement boundaries. The reported metric is kernel-only CUDA-event time and excludes setup, object construction, data movement, copyback, and checksum code. This isolates the cost of gPAC instrumentation in the main compute region, but it is not a whole-application wall-clock metric. Full performance accounting should add CUPTI or Nsight Compute measurements for dynamic instruction counts, occupancy, stalls, and memory-system effects.

9 Conclusion

gPAC addresses a practical gap in CUDA device-code protection: selected memory-resident control pointers can be corrupted by GPU memory-safety bugs, but commodity NVIDIA GPUs do not provide hardware pointer authentication for device code, and full memory-safety defenses may be too broad for performance-sensitive kernels. gPAC provides low-overhead, software-enforced probabilistic integrity for device function pointers and object-resident vptrs by embedding compact tags in unused pointer high bits, deriving authentication state from hidden per-kernel-instance launch seeds, and checking protected values before indirect device control transfer. Its guarantee is intentionally bounded by tag width, key secrecy, replay assumptions, and the selected authentication backend; it is not complete memory safety or hardware-equivalent PAC. Within that boundary, our LLVM 18-based implementation runs all 18 evaluated CUDA workloads and incurs 3.59% median kernel-only overhead with a $1.042\times$ geometric-mean slowdown.

References

1. Miele, A.: Buffer overflow vulnerabilities in CUDA: a preliminary analysis. arXiv:1506.08546 (2015). <https://arxiv.org/abs/1506.08546>
2. Di, B., Sun, J., Chen, H.: A Study of Overflow Vulnerabilities on GPUs. In: 13th IFIP WG 10.3 International Conference on Network and Parallel Computing (NPC), pp. 103–115. Springer (2016). https://doi.org/10.1007/978-3-319-47099-3_9
3. Guo, Y., Zhang, Z., Yang, J.: GPU Memory Exploitation for Fun and Profit. In: 33rd USENIX Security Symposium (USENIX Security), pp. 4033–4050. USENIX Association (2024). <https://www.usenix.org/conference/usenixsecurity24/presentation/guo-yanan>
4. Roels, J., Jacobs, A., Volckaert, S.: CUDA, Woulda, Shoulda: Returning Exploits in a SASS-y World. In: 18th European Workshop on Systems Security (EuroSec), pp. 40–48. ACM (2025). <https://doi.org/10.1145/3722041.3723099>
5. NVIDIA Corporation: Compute Sanitizer Documentation. <https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html>
6. Erb, C., Collins, M., Greathouse, J.L.: Dynamic Buffer Overflow Detection for GPGPUs. In: IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 61–73 (2017). <https://doi.org/10.1109/CGO.2017.7863729>
7. Di, B., Sun, J., Chen, H., Li, D.: Efficient Buffer Overflow Detection on GPU. IEEE Transactions on Parallel and Distributed Systems 32(5), 1161–1177 (2021). <https://doi.org/10.1109/TPDS.2020.3042965>
8. Lee, J., Kim, Y., Cao, J., Kim, E., Lee, J., Kim, H.: Securing GPU via Region-Based Bounds Checking. In: 49th Annual International Symposium on Computer Architecture (ISCA), pp. 27–41. ACM (2022). <https://doi.org/10.1145/3470496.3527420>
9. Ibn Ziad, M.T., Damani, S., Jaleel, A., Keckler, S.W., Stephenson, M.: cuCatch: A Debugging Tool for Efficiently Catching Memory Safety Violations in CUDA Applications. Proceedings of the ACM on Programming Languages 7(PLDI), Article 111, 124–147 (2023). <https://doi.org/10.1145/3591225>

10. Zhang, C., Hou, R.: LAK: A Low-Overhead Lock-and-Key Based Schema for GPU Memory Safety. In: IEEE 40th International Conference on Computer Design (ICCD), pp. 705–713 (2022). <https://doi.org/10.1109/ICCD56317.2022.00108>
11. Sullivan, M.B., Ibn Ziad, M.T., Jaleel, A., Keckler, S.W.: Implicit Memory Tagging: No-Overhead Memory Safety Using Alias-Free Tagged ECC. In: 50th Annual International Symposium on Computer Architecture (ISCA). ACM (2023). <https://doi.org/10.1145/3579371.3589102>
12. Ibn Ziad, M.T., Damani, S., Stephenson, M., Keckler, S.W., Jaleel, A.: GPUArmor: A Hardware-Software Co-design for Efficient and Scalable Memory Safety on GPUs. arXiv:2502.17780 (2025). <https://arxiv.org/abs/2502.17780>
13. Santos-Grueiro, I.: WarpGuard: Protected-Site Control-Flow Integrity for CUDA SASS Binaries. arXiv:2606.11871 (2026). <https://arxiv.org/abs/2606.11871>
14. Abadi, M., Budiu, M., Erlingsson, Ú., Ligatti, J.: Control-Flow Integrity. In: 12th ACM Conference on Computer and Communications Security (CCS), pp. 340–353. ACM (2005). <https://doi.org/10.1145/1102120.1102165>
15. Tice, C., Roeder, T., Collingbourne, P., Checkoway, S., Erlingsson, Ú., Lozano, L., Pike, G.: Enforcing Forward-Edge Control-Flow Integrity in GCC & LLVM. In: 23rd USENIX Security Symposium (USENIX Security), pp. 941–955. USENIX Association (2014). <https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/tice>
16. The LLVM Project: Control Flow Integrity. Clang documentation. <https://clang.llvm.org/docs/ControlFlowIntegrity.html>
17. van der Veen, V., Andriesse, D., Göktas, E., Gras, B., Sambuc, L., Slowinska, A., Bos, H., Giuffrida, C.: Practical Context-Sensitive CFI. In: 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 927–940. ACM (2015). <https://doi.org/10.1145/2810103.2813673>
18. Kuznetsov, V., Szekeres, L., Payer, M., Candea, G., Sekar, R., Song, D.: Code-Pointer Integrity. In: 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pp. 147–163. USENIX Association (2014). <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/kuznetsov>
19. Evans, I., Fingeret, S., Gonzalez, J., Otgonbaatar, U., Tang, T., Shrobe, H., Sidiroglou-Douskos, S., Rinard, M., Okhravi, H.: Missing the Point(er): On the Effectiveness of Code Pointer Integrity. In: IEEE Symposium on Security and Privacy (S&P), pp. 781–796 (2015). <https://doi.org/10.1109/SP.2015.53>
20. Mashtizadeh, A.J., Bittau, A., Boneh, D., Mazières, D.: CCFI: Cryptographically Enforced Control Flow Integrity. In: 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 941–951. ACM (2015). <https://doi.org/10.1145/2810103.2813676>
21. Arm Limited: Arm Architecture Reference Manual for A-profile architecture. <https://developer.arm.com/documentation/ddi0487/latest>
22. Liljestrand, H., Nyman, T., Wang, K., Chinae Perez, C., Ekberg, J.-E., Asokan, N.: PAC it up: Towards Pointer Integrity using ARM Pointer Authentication. In: 28th USENIX Security Symposium (USENIX Security), pp. 177–194. USENIX Association (2019). <https://www.usenix.org/conference/usenixsecurity19/presentation/liljestrand>
23. Liljestrand, H., Nyman, T., Gunn, L.J., Ekberg, J.-E., Asokan, N.: PACStack: an Authenticated Call Stack. In: 30th USENIX Security Symposium (USENIX Security), pp. 357–374. USENIX Association (2021). <https://www.usenix.org/conference/usenixsecurity21/presentation/liljestrand>

24. Farkhani, R.M., Ahmadi, M., Lu, L.: PTAAuth: Temporal Memory Safety via Robust Points-to Authentication. In: 30th USENIX Security Symposium (USENIX Security), pp. 1037–1054. USENIX Association (2021). <https://www.usenix.org/conference/usenixsecurity21/presentation/mirzazade>
25. Li, Y., Tan, W., Lv, Z., Yang, S., Payer, M., Liu, Y., Zhang, C.: PACMem: Enforcing Spatial and Temporal Memory Safety via ARM Pointer Authentication. In: ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 1901–1915. ACM (2022). <https://doi.org/10.1145/3548606.3560598>
26. The LLVM Project: Hardware-assisted AddressSanitizer Design Documentation. Clang documentation. <https://clang.llvm.org/docs/HardwareAssistedAddressSanitizerDesign.html>
27. Arm Limited: Armv8.5-A Memory Tagging Extension White Paper. <https://developer.arm.com/documentation/102925/0100/>
28. Intel Corporation: Intel 64 and IA-32 Architectures Software Developer’s Manual. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
29. Jang, D., Tatlock, Z., Lerner, S.: SafeDispatch: Securing C++ Virtual Calls from Memory Corruption Attacks. In: Network and Distributed System Security Symposium (NDSS) (2014). <https://www.ndss-symposium.org/ndss2014/ndss-2014-programme/safedispatch-securing-c-virtual-calls-memory-corruption-attacks/>
30. Prakash, A., Hu, X., Yin, H.: vfGuard: Strict Protection for Virtual Function Calls in COTS C++ Binaries. In: Network and Distributed System Security Symposium (NDSS) (2015). <https://www.ndss-symposium.org/ndss2015/ndss-2015-programme/vfguard-strict-protection-virtual-function-calls-cots-c-binaries/>
31. Zhang, C., Carr, S.A., Li, T., Ding, Y., Song, C., Payer, M., Song, D.: VTrust: Regaining Trust on Virtual Calls. In: Network and Distributed System Security Symposium (NDSS) (2016). <https://doi.org/10.14722/ndss.2016.23164>
32. Burow, N., McKee, D.P., Carr, S.A., Payer, M.: CFIXX: Object Type Integrity for C++. In: Network and Distributed System Security Symposium (NDSS) (2018). <https://doi.org/10.14722/ndss.2018.23279>
33. Chen, K., Zhang, C., Yin, T., Chen, X., Zhao, L.: VScape: Assessing and Escaping Virtual Call Protections. In: 30th USENIX Security Symposium (USENIX Security), pp. 1719–1736. USENIX Association (2021). <https://www.usenix.org/conference/usenixsecurity21/presentation/chen-kaixiang>
34. NVIDIA Corporation: CUDA C++ Programming Guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
35. Zhang, M., Alawneh, A., Rogers, T.G.: Judging a Type by Its Pointer: Optimizing GPU Virtual Functions. In: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 241–254. ACM (2021). <https://doi.org/10.1145/3445814.3446734>